



Running Underworld in a Browser: Any Repository, Any Version

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Python/Jupyter

Somebody reads a paper, wants to run the model, and has forty minutes. They will not install PETSc. They may not have a compiler. If the answer is “clone this, then build these dependencies”, the answer is really “no”.

The answer we have now is a link. It opens JupyterLab in a browser, with Underworld already built, **any public repository** pulled in beside it, and **any released version** of Underworld underneath. Those three choices are independent, and the repository being launched needs nothing added to it — no Dockerfile, no `.binder/` directory, no configuration at all.

This note is about how that works, because the interesting parts are not obvious and one of them is specific to a code that compiles itself at runtime.

1. WHERE THIS CAME FROM: THE CLASSROOM

The forty-minute reader is the general case, but the case that drove the work was teaching, and it is a harder problem than it looks.

A two-hour practical with thirty students has, historically, been spent as follows: forty minutes installing, forty minutes on the six laptops where the install went wrong, and the remainder on the geodynamics. Departmental lab machines fix this until you need a version they do not have, or a student wants to continue at home. Underworld’s own cloud was built to escape that loop and it did, at the cost of somebody running a Kubernetes cluster.

What a class actually needs turns out to be modest:

- **Nothing installed.** A browser, on whatever the student owns.
- **Everyone on the same version**, all semester. If the practicals were written against `v3.1.0`, then `v3.1.0` is what they run in week nine, no matter what happened on `development` in the meantime. This is the requirement that a plain “latest” link cannot meet, and it is why release branches are frozen.
- **One link per practical**, each opening the folder for that week, so nobody is navigating a file tree to find where they are supposed to be.

- **Corrections that take effect immediately.** Fix the notebook, push, and the next student to click gets the fixed one — no reissued handout.

The last two fall out of the URL described below; the second falls out of the release machinery. That is the whole design brief, and it is why the pieces are arranged the way they are.

Below university level, the calculation changes. A school has no departmental cluster and no expectation of one, and often no ability to install anything on a managed device. But a link is not software — it is a link, and it opens the same way a video does. Some of what Underworld produces is legible well before undergraduate level: a fault slipping and the ground deforming around it, a slab sinking, plates pulling apart. A class that could never be asked to install a finite element code can be asked to click something and change a number. We have not built materials pitched that way; the obstacle was never the delivery, and now the delivery is free.

2. THE SHAPE OF IT

Four pieces, each doing one job:

1. A **container image** with Underworld already built, published to the GitHub Container Registry.
2. A **launcher repository** — almost empty — that mybinder.org caches against.
3. Two **GitHub workflows** that build the image on a release and, in the same run, create the launcher branch that points at it.
4. **nbgitpuller**, which clones the reader's repository into the running session.

The version guarantee comes from (3): the release and its launcher are made together, so they cannot drift apart.

3. THE CONTAINER, AND WHAT MAY NOT BE REMOVED FROM IT

The image is built in stages and then stripped, because binder start-up time is dominated by pulling it. The savings that mattered:

Removed	Saved
docs_legacy	229 MB
pixi package cache	~500 MB
conda-meta metadata	24 MB
man pages, __pycache__, *.pyc, test suites	tens of MB

The git clone is `--depth 1 --single-branch`, which keeps `.git` at about 5 MB instead of 356 MB. It is kept rather than deleted, because a shallow history is still enough to `git pull` at start-up.

There is also a layer-size problem worth knowing about if you ever build one of these. The runtime library directory is around 2.7 GB, and a single Docker layer that large is unwieldy to push and pull. So the libraries are split by family — LLVM, VTK, gmsh, OpenBLAS, Qt — and copied in chunks under 800 MB, so no layer is over a gigabyte.

And then the part that is specific to Underworld. The obvious next economies are to delete the compiler toolchain and the C header files, which between them are substantial and which no ordinary Python image needs after the build. Underworld cannot. It turns symbolic mathematics into C and compiles it *while the model*

runs — that is the whole design, and it is the subject of a [note of its own](#). Strip the compiler and the image builds, imports, and then fails the moment a user defines a constitutive model.

So the Dockerfile carries these lines, commented out, with the reason attached:

```
# KEEP include directory - needed for JIT compilation at runtime
# KEEP compiler toolchain - needed for JIT compilation at runtime
```

A container for a JIT-compiling code is not a container for a Python package. It has to ship the means of production, not just the product.

4. THE LAUNCHER: AN ALMOST EMPTY REPOSITORY

`underworldcode/uw3-binder-launcher` contains, per branch, a `.binder/Dockerfile` of two meaningful lines:

```
FROM ghcr.io/underworldcode/uw3-base:v3.1.0-slim
ENV UW3_BRANCH=v3.1.0
```

That is the whole thing. Why does it exist at all, rather than pointing binder at the Underworld repository?

Because mybinder caches on the commit hash of the repository you launch. If you launch a repository that changes daily, you miss the cache daily, and every miss is a full image build in front of a waiting reader. The launcher's job is to be a repository that almost never changes — so the cache almost always hits, and the Underworld code arrives as a pre-built image rather than being built on demand.

This is why a first launch after a release is slow and every launch after that is quick. Not a fault: a cache being filled.

5. THE WORKFLOWS THAT KEEP VERSIONS HONEST

In the Underworld repository, `binder-image.yml` triggers on a push to `main` or `development`, on any `v*` tag, and on changes to the Dockerfile, the pixi lock file, or any Cython source — the things that actually require a rebuild. It builds the image, pushes it to GHCR tagged for the branch or release, and then does the thing that matters:

```
- name: Trigger launcher update
  uses: peter-evans/repository-dispatch@v2
  with:
    repository: underworldcode/uw3-binder-launcher
    event-type: image-updated
    client-payload: '{"branch": "...", "ref_type": "..."}'
```

In the launcher repository, `update-image.yml` listens for that and behaves differently according to what arrived:

- A **branch push** updates the existing launcher branch's `Dockerfile` to point at the new image. `main` and `development` therefore track.
- A **release tag** creates a *new launcher branch* named for the tag, containing a frozen `Dockerfile` pinned to that release's image.

The word to notice is *frozen*. A release branch is written once and then nothing changes it. `v0.99` will still be `v0.99` in five years, because there is no process that would rewrite it — and no human step that could be forgotten.

6. NBGITPULLER: YOUR NOTEBOOKS, NOTHING REQUIRED OF THEM

The launcher image carries `nbgitpuller`, which clones a repository into the session at start-up and merges updates on later launches. The consequences are the useful part:

- Your repository needs **no** binder configuration. The environment comes from the launcher; only the notebooks come from you.
- It is pulled **fresh on every launch**, so a correction you push is live for the next person who clicks.
- The requirements are: public on GitHub, notebooks using the `python3` kernel, and `import underworld3 as uw`.

7. THE URL, TAKEN APART

Written plainly, the link says: which Underworld, which repository, and where to open.

```
https://mybinder.org/v2/gh/underworldcode/uw3-binder-launcher/VERSION
?urlpath=git-pull
&repo=https://github.com/USER/REPO
&branch=BRANCH
&urlpath=lab/tree/REPO/WHERE
```

Part	What it selects
VERSION	the launcher branch: <code>main</code> , <code>development</code> , or a release such as <code>v3.1.0</code>
repo	the repository to clone alongside Underworld
branch	which branch of it
the second urlpath	where JupyterLab opens: a folder, or one notebook

Two `urlpath` parameters is not a mistake. The first tells binder to hand over to `nbgitpuller`; the second is `nbgitpuller`'s own instruction about where to land once it has finished cloning.

The escaping. That plain form is not what you paste. It is a URL nested inside a URL, so everything after `git-pull` must be percent-encoded — and the repository address, one level deeper again, is encoded twice. `/` becomes `%2F` at one level and `%252F` at two. This is why the working links look the way they do, and why writing one by hand is a poor use of an afternoon:

```
python scripts/binder_wizard.py myuser/my-course main tutorials/intro.ipynb
```

That emits the encoded URL and a ready-to-paste badge in Markdown, HTML or reStructuredText — which is how a **Launch** button gets onto a course or paper repository.

8. SETTING UP A COURSE

Put the practicals in one public repository, a folder per week:

```
geodynamics-2026/
  week-01-convection/
  week-02-rheology/
  week-03-subduction/
```

Then issue one link per week, identical apart from the folder, and all naming the same release:

```
.../uw3-binder-launcher/v3.1.0?...&urlpath=lab/tree/geodynamics-2026/week-01-convection
.../uw3-binder-launcher/v3.1.0?...&urlpath=lab/tree/geodynamics-2026/week-02-rheology
```

Nothing else is needed: no accounts, no lab image, no install instructions, and no version drift over the semester. A fix pushed on Tuesday is what the Wednesday group gets.

One practical caution. mybinder.org is free and shared, and thirty simultaneous launches is a real load on it. The cache works in your favour — the first launch pulls the image and the rest are quick — so it is worth clicking the link yourself an hour before the class to make sure the image is warm. And if the service is busy or down, the practical is down. For an assessed session, have the notebooks runnable locally as a fallback, or use a JupyterHub you control; the same launcher image works there.

For a class whose work must persist between sessions, remember that these sessions do not. Have students push to their own repository, or download at the end — which is a reasonable thing to teach anyway.

9. WHAT THIS DOES AND DOES NOT GUARANTEE

It guarantees the **environment**. Pinning to `v3.1.0` fixes Underworld, its dependencies, and the compiler that builds its generated C. A notebook that ran then will run now.

It does not fix your data. A notebook that downloads a dataset at run time is only as reproducible as that download, and no container can help. If it matters, put the data in the repository.

Three other limits, stated plainly because they are the trade for not running servers:

- **Sessions are ephemeral.** There is no home directory. Push your work to git or download it before closing the tab.
- **mybinder.org is a free, shared service.** It is busy sometimes, and it has memory and CPU limits. It is for teaching, demonstrating and trying things — not for production runs.
- **Public repositories only,** because there is nowhere to put a credential.

For anything past that, install Underworld or run it on a cluster.

10. A NOTE ON WHAT THIS REPLACED

Underworld used to run its own cloud — Kubernetes for large classes, single droplets for small ones, under an [AuScope](#) project. It was built for exactly the classroom problem above, it solved it, and it did one thing this does not: it gave every user a persistent home directory, which for a semester-long course is a genuine loss.

What it also did was require somebody to run it, and pay for it, and be available when it broke on a Tuesday morning. The arrangement described here does the same job with no servers, no cost and no operator, and

adds the version pinning the old one never had — which for teaching matters more than the home directory did. That is the trade, and it is why the cloud has been retired.